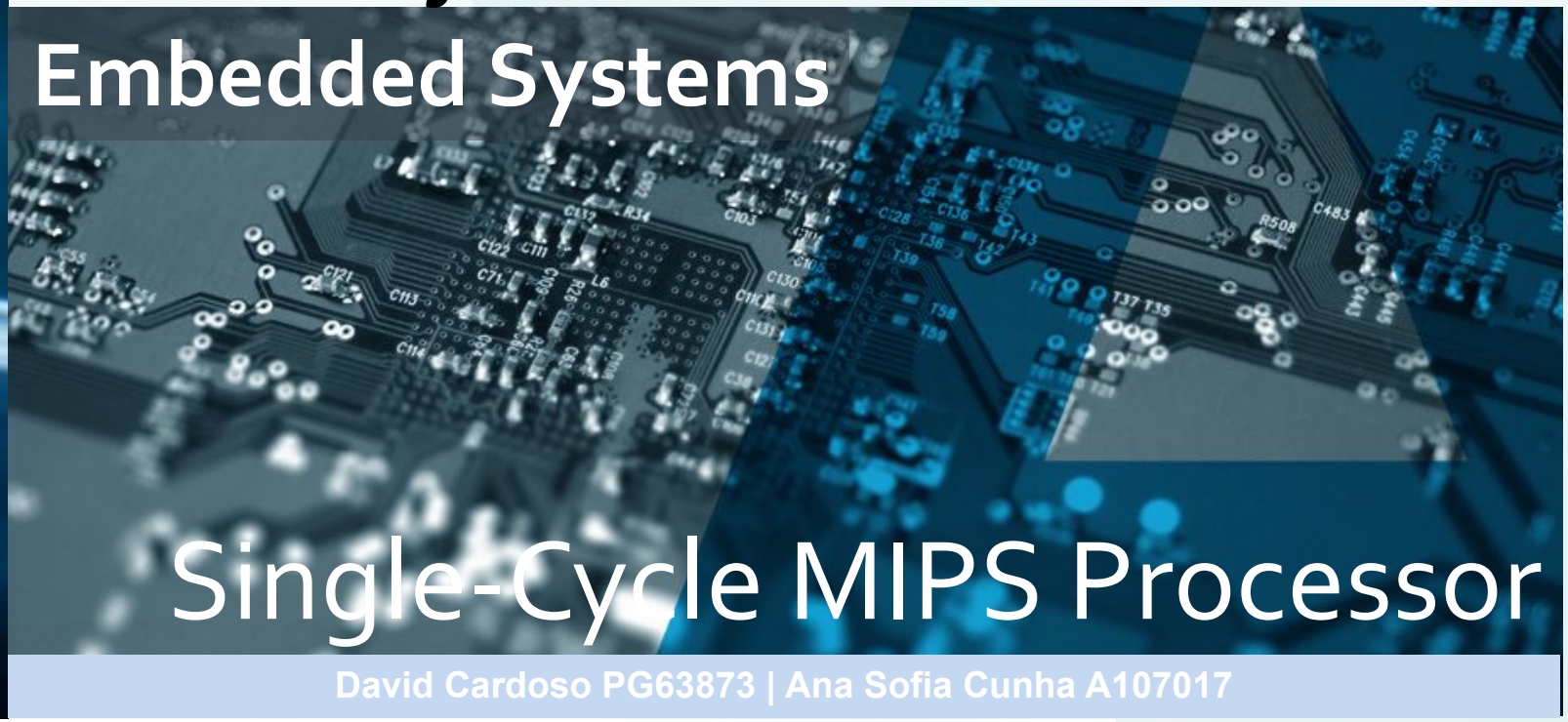
ESRG
Embedded Systems
Research Group

CENTROALGORITMI

MIPS Project

Embedded Systems



Single-Cycle MIPS Processor

David Cardoso PG63873 | Ana Sofia Cunha A107017

Universidade do Minho
Escola de Engenharia

COMPETE

QREN Quadro de Referência Estratégico Nacional

União Europeia
Fundo Europeu de Desenvolvimento Regional

FCT Fundação para a Ciência e a Tecnologia
Ministério da Ciência, Tecnologia e Ensino Superior

Agenda

- Project Overview
- Zybo Z7-10
- Instruction Formats
- Instruction Set Architecture (ISA)
- Register File
- Control Unit
- ALU Operations
- Next Step

Project Overview

1. Specify: 32-bit single-cycle MIPS, 16-instruction base ISA agreed by the class, instruction encoding and datapath

2. Build: each module in Verilog (ALU, register file, control unit, memories), with its own testbench

3. Integrate and demonstrate: full processor simulated in Vivado, then running a test program on the Zybo Z7-10 FPGA

4. Extend: additional instructions, memory-mapped I/O, timer and interrupts

Current status: step 1 (specification and datapath design)

Zybo Z7-10

Target Platform: Digilent Zybo Z7-10

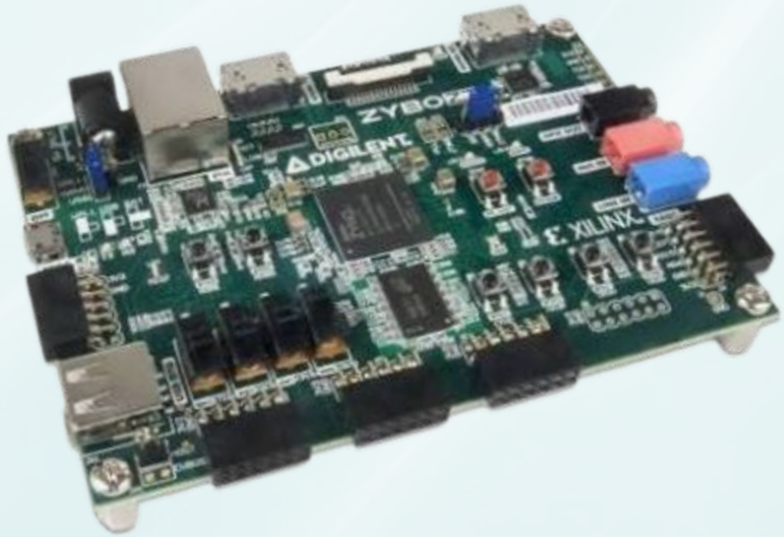
Device: Xilinx Zynq-7000 (XC7Z010-1CLG400C)

What we use: only the Programmable Logic (FPGA). The ARM cores are not used.

Resources: 17,600 LUTs | 35,200 flip-flops | 270 KB Block RAM

Board I/O: 125 MHz clock, push-buttons, slide switches and LEDs

Tool: Vivado Design Suite



Instruction Formats

Format	Fields
R-Type	opcode (31 - 26) | rs (25 - 21) | rt (20 -16) | rd (15 - 11) | shamt (10 -6) | funct (5 - 0)
I-Type	opcode (31 - 26) | rs (25 - 21) | rt (20 -16) | immediate (15 - 0)
J-Type	opcode (31 -26) | target address (25 - 0)

Examples:

- R-Type: and $to, $t1, $t2 -> and rd, rs, rt
- I-Type: addi $to, $t1, 1 → addi rt, rs, immediate
- J-Type: j target → j target address

Instruction Set Architecture (ISA)

Class-wide ISA

Format	*Instructions*
R-Type	add, sub, and, or, xor, slt, sll, srl, jr
I-Type	addi, lw, sw, beq, bne
J-Type	j, jal

Additional Instructions

Format	*Instructions*
R-Type	nor, sltu
I-Type	lui, ori, andi, xori
J-Type	-
COP0	eret

Instruction Set Architecture (ISA)

R-Type

Instr.	opcode (31–26)	rs (25–21)	rt (20–16)	rd (15–11)	shamt (10–6)	funct (5–0)
add	000000	rs	rt	rd	00000	100000
sub	000000	rs	rt	rd	00000	100010
and	000000	rs	rt	rd	00000	100100
or	000000	rs	rt	rd	00000	100101
xor	000000	rs	rt	rd	00000	100110
nor	000000	rs	rt	rd	00000	100111
slt	000000	rs	rt	rd	00000	101010
sltu	000000	rs	rt	rd	00000	101011
sll	000000	00000	rt	rd	shamt	000000
srl	000000	00000	rt	rd	shamt	000010
jr	000000	rs	00000	00000	00000	001000

Instruction Set Architecture (ISA)

I-Type

Instr.	opcode (31–26)	rs (25–21)	rt (20–16)	immediate (15–0)
addi	001000	rs	rt	immediate
lw	100011	rs	rt	offset
sw	101011	rs	rt	offset
beq	000100	rs	rt	offset
bne	000101	rs	rt	offset
andi	001100	rs	rt	immediate
ori	001101	rs	rt	immediate
xori	001110	rs	rt	immediate
lui	001111	00000	rt	immediate

Instruction Set Architecture (ISA)

J-Type

Instr.	opcode (31 – 26)	target (25 – 0)
j	000010	target[25:0]
jal	000011	target[25:0]

Operation COPo

Instr.	opcode (31–26)	CO (25)	(24 - 6)	funct (5 - 0)
eret	010000	1	00…00	011000

COPo instruction (R-type-like layout), encoding 0x42000018, decoded by opcode 010000.

Instruction Set Architecture (ISA)

- **lui, ori:** load any 32-bit constant in two instructions;
- **andi, ori, xori:** test, set and toggle bits with immediates;
- **nor:** NOT operation and clearing bits in 32-bit values;
- **sltu:** correct comparison of unsigned values;
- **eret:** atomic hardware return from interrupts (PC = EPC and re-enables interrupts). EPC: 32-bit register that saves the return address on interrupt entry; IE: interrupt-enable flag.

Register File

- 32 registers × 32 bits
- 2 asynchronous read ports (rs, rt)
- 1 synchronous write port, written on the clock edge when RegWrite = 1
- $zero always reads 0; writes to it are ignored
- The write register can be rt (I-type), rd (R-type) or $31 (jal), so the RegDst mux has 3 inputs
- The write data can come from the ALU, memory or PC+4 (jal)

Name	No.	Use
$zero	0	Constant 0 (hardwired)
$at	1	Assembler temporary
$v0–$v1	2–3	Return values
$a0–$a3	4–7	Function arguments
$t0–$t7	8–15	Temporaries
$s0–$s7	16–23	Saved registers
$t8–$t9	24–25	Temporaries
$k0–$k1	26–27	Reserved (kernel)
$gp	28	Global pointer
$sp	29	Stack pointer
$fp	30	Frame pointer
$ra	31	Return address

Control Unit

Instruction	RegWrite	RegDst	ALUSrc	ExtOp	MemWrite	WBSrc	IE	PCSrc
add, sub, and, or, xor, nor, slt, sltu	1	rd	reg	-	0	ALU	-	PC+4
sll, srl	1	rd	reg	-	0	ALU	-	PC+4
addi	1	rt	imm	sign	0	ALU	-	PC+4
andi, ori, xori, lui	1	rt	imm	zero	0	ALU	-	PC+4
lw	1	rt	imm	sign	0	Mem	-	PC+4
sw	0	-	imm	sign	1	-	-	PC+4
beq, bne	0	-	reg	sign	0	-	-	branch
j	0	-	-	-	0	-	-	jump
jal	1	$31	-	-	0	PC+4	-	jump
jr	0	-	-	-	0	-	-	rs
eret	0	-	-	-	0	-	1	EPC

Next PC Logic

PCSrc	Next PC	Used by
000	PC + 4	sequential execution
001	(PC+4) + (SignExt(imm) << 2)	beq, bne (if taken)
010	{(PC+4)[31:28], index, 00}	j, jal
011	rs	jr
100	EPC	eret
101–111	reserved	interrupt vector (Phase 2)

- *eret adds a 5th PC source (EPC), so PCSrc grows from 2 to 3 bits.*
- *Branches select the branch target only when taken (beq: zero = 1, bne: zero = 0).*
- *Free codes leave room for the interrupt vector without changing the mux.*

ALU Operations

ALUCtrl	Operation	Function	Used by
0000	AND	A & B	and, andi
0001	OR	A | B	or, ori
0010	ADD	A + B	add, addi, lw, sw
0011	XOR	A ^ B	xor, xori
0100	SLL	B << shamt	sll
0101	SRL	B >> shamt	srl
0110	SUB	A - B	sub, beq, bne
0111	SLT	A < B (signed)	slt
1000	NOR	~(A | B)	nor
1001	SLTU	A < B (unsigned)	sltu
1010	LUI	B << 16	lui
1011-1111	free	reserved for extensions	-

Next Step

1. ISA Implementation & Verification

- Build the single-cycle datapath components: PC, register file, ALU, sign/zero extender, and instruction and data memories.
- Design the main control unit and ALU control for all 23 instructions (class-wide ISA plus lui, ori, andi, xori, sltu, nor and eret).
- Verify standard ISA compliance and execution accuracy through behavioral simulations using assembly testbenches.

Thank you for your attention!